

Modules 17–18: Prompt Engineering › Retrieval-Augmented Generation (RAG)

PROGRAM	ASSESSMENT CODE	TOTAL TIME	STACK
Data Science	M18-A1	3 Hours	Python 3

GENERAL INSTRUCTIONS

Attempt all sections. Write your code in Python 3 unless stated otherwise. For coding tasks, submit your .py source file along with a screenshot of your program output. Label every task clearly with its section and task number. This is a cumulative assessment covering Module 17 (Prompt Engineering) and Module 18 (Retrieval-Augmented Generation). All work must be your own.

A Section A — Concept Application

Answer all questions. Each question presents a real-world scenario followed by a concept-specific question.

SCENARIO

**S
1**

You are building a customer support chatbot for a food delivery app. During testing, you notice that the chatbot gives inconsistent answers — sometimes it refuses valid refund requests, and sometimes it approves them without verifying order status. Your team traces the problem to vague prompts that do not specify the expected output format or the decision rules the LLM should follow.

Question: Explain what distinguishes a clear prompt from an unclear one in this context. Describe at least two specific changes you would make to the existing prompts to produce consistent, reliable responses from the LLM, and justify why each change matters.

SCENARIO

**S
2**

You are designing a complaint classifier for a food delivery platform. The classifier must label incoming customer messages into one of four categories: 'Late Delivery', 'Wrong Item', 'Missing Item', and 'Poor Quality'. You have 200 labelled examples available, but the LLM you are using has a limited context window and cannot accept all of them at once.

Question: Compare zero-shot and few-shot prompting as strategies for building this classifier. Justify which approach you would choose given the constraints described, and explain what factors determine how many examples to include in a few-shot prompt.

SCENARIO**S
3**

You are developing an AI assistant that helps delivery operations staff calculate the optimal stop sequence for a delivery agent given five order addresses, estimated traffic delays, and priority levels. When you send a direct question to the LLM asking for the best route, the answers are frequently incorrect and contain no explanation of the reasoning.

Question: Explain how Chain-of-Thought (CoT) prompting would improve the quality of the assistant's output for this task. Write a brief example showing how you would restructure the prompt to invoke step-by-step reasoning, and identify one limitation of this technique.

SCENARIO**S
4**

You are working on an internal tool that lets delivery partners query a 150-page policy handbook — covering payout rules, SLA terms, and dispute resolution — that is updated every quarter. A colleague proposes fine-tuning an LLM on the handbook, while you propose building a RAG system instead.

Question: Justify your choice of RAG over fine-tuning for this specific use case. Address at least two reasons, including how each approach handles frequent document updates, and identify one scenario in which fine-tuning would have been the better choice.

SCENARIO**S
5**

You are building a RAG pipeline to let customers search across 500 restaurant menus stored as PDF files. Each menu averages 8 pages. During evaluation you find that retrieved chunks often cut mid-sentence or omit critical context — for example, a price appears in a sentence one paragraph after the dish name, but both rarely appear in the same chunk.

Question: Explain the role of chunk size and chunk overlap in a RAG pipeline. Describe how you would adjust these two parameters to fix the retrieval quality problems described, and identify one trade-off your adjustment introduces.

SCENARIO

S
6

You are optimising the retrieval component of a RAG-based complaint resolution system for a food delivery platform. The system stores complaint summaries as embeddings in a FAISS index. Query speed is excellent, but retrieved results sometimes include outdated records for resolved complaints that should have been removed. A team member suggests switching to ChromaDB.

Question: Compare FAISS and ChromaDB as vector store options. Explain one scenario where each is the better fit, evaluate whether switching to ChromaDB would solve the outdated-record problem described here, and identify what additional step would be required regardless of which vector store is used.

B

Section B — Practical Coding Tasks

Complete all tasks. Each task targets one specific concept or feature from the module. Follow the requirements listed under each task.

Task 1: Structured Prompt Builder for Food Delivery Chatbot

Build a Python script that constructs and validates a role-based, structured prompt for a food delivery customer support chatbot and prints the formatted prompt to the console.

- Define a system prompt string that assigns the LLM the role of a food delivery support agent, sets a professional and helpful tone, and specifies that responses must not exceed 80 words.
- Define a user message as a formatted string template that accepts three variables: `customer_name`, `order_id`, and `issue_type`; populate the template with at least two different test cases and print the full system + user prompt pair for each.
- Add a validation function that raises a `ValueError` with a clear message if `issue_type` is not one of three allowed values: 'late delivery', 'missing item', or 'wrong item'.
- Wrap the prompt construction in a try-except block that catches the `ValueError` and prints a user-friendly error message without crashing the script.

Task 2: Few-Shot Complaint Classifier Prompt Builder

Build a Python function that constructs a few-shot prompt for classifying food delivery complaints into predefined categories and returns the formatted prompt string ready for submission to an LLM API.

- Define a list of at least four labelled examples in Input / Output format, covering all four complaint categories: 'Late Delivery', 'Wrong Item', 'Missing Item', and 'Poor Quality'.

- Write a function `build_few_shot_prompt(complaint_text)` that inserts the labelled examples into the prompt before appending the new complaint and a blank Output line for the LLM to complete.
- Write a second function `add_example(text, label)` that appends a new labelled pair to the examples list; call it once with a new example, then call `build_few_shot_prompt` again and confirm the new example appears in the printed output.
- Test the full prompt builder with at least two different complaint strings and print the resulting prompt for each, showing the consistent structure across calls.

Task 3: Semantic Search Over Restaurant FAQs

Build a Python script that encodes a set of restaurant FAQ entries into embeddings using sentence-transformers and retrieves the most semantically relevant FAQ for a given user query using a FAISS index.

- Define at least six FAQ strings covering common food delivery topics such as delivery time, order cancellation, refund eligibility, item substitution, and contact support; generate embeddings for all FAQs using `SentenceTransformer('all-MiniLM-L6-v2')`.
- Build a FAISS `IndexFlatL2` index from the FAQ embeddings converted to a float32 NumPy array and confirm the index size matches the number of FAQs.
- Write a function `search_faq(query, k=2)` that encodes the query, searches the index, and returns the top-k most relevant FAQ strings along with their L2 distances.
- Test the function with at least two queries that use different words from the stored FAQs (e.g. 'How do I get money back?' vs an FAQ that says 'Refund policy') and print the results to confirm that semantic matching works beyond exact keyword overlap.

Task 4: RAG-Powered Food Delivery Policy Q&A Pipeline

Build a Python script that implements a complete minimal RAG pipeline — chunking a policy document, embedding the chunks, retrieving relevant context, and assembling a structured final prompt — to answer user questions about food delivery policies.

- Define a multi-paragraph food delivery policy text of at least 300 words covering refund rules, delivery windows, and order cancellation; split it into overlapping chunks of approximately 100 words with a 20-word overlap and print the total chunk count.
- Generate sentence-transformer embeddings for all chunks, store them in a FAISS `IndexFlatL2` index, and implement a `retrieve(query, k=3)` function that returns the top-k most relevant chunk strings.
- Write a `build_rag_prompt(query, retrieved_chunks)` function that constructs a prompt in the following order: system role instruction › numbered retrieved context blocks › user question › explicit instruction to answer only from the provided context and to say 'I don't know' if the answer is not present.
- Demonstrate the full end-to-end pipeline with a hardcoded question such as 'What is the refund policy for missing items?' — print the retrieved chunks, the assembled prompt, and a placeholder simulated answer string.

C

Section C — Mini Capstone Project

Mini Project: Food Delivery Help Desk Chatbot (RAG-Powered Console App)

Objective:

Build a console-based help desk chatbot for a food delivery platform that combines document chunking, FAISS-based retrieval, structured prompt engineering, and few-shot complaint classification into a single interactive program with session logging.

Your project must:

- The program must be menu-driven with at least three options: (1) Ask a policy question via RAG, (2) Classify a complaint using few-shot prompting, (3) Exit — and must loop until the user selects Exit.
- For option 1, load and chunk a predefined food delivery policy text (minimum 500 words covering refund, cancellation, and delivery rules) at startup, embed all chunks using sentence-transformers, and build a FAISS index; on each query, retrieve the top-3 relevant chunks, assemble a structured RAG prompt, and print both the retrieved context and the complete prompt.
- For option 2, maintain a few-shot example bank of at least four labelled complaints; accept a user-typed complaint, build and print the few-shot classification prompt, and display the predicted category along with the closest matching example from the bank.
- Validate all user inputs: reject empty questions, reject blank complaint text, display a user-friendly error message, and re-prompt without crashing.
- Maintain a session log that writes each interaction — timestamp, mode (RAG / Classify), query text, and output — to a plain-text file; on exit, print a summary showing the total number of policy questions asked and complaints classified during the session.

D

Section D — AI-Augmented Learning

STEP 1 • BUILD WITH AI

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a program that:

- Reads a plain-text food delivery refund policy file from disk, splits it into fixed-size chunks, generates embeddings using sentence-transformers, and stores them in a FAISS index.
- Accepts a user question typed at the command line, retrieves the top-2 most relevant chunks from the index, and constructs a structured RAG prompt using those chunks as context.
- Prints the retrieved chunks, the assembled prompt, and a simulated placeholder answer (a hardcoded string is acceptable — no live API call is required).
- Runs in a loop so the user can ask multiple questions without restarting the script, and exits cleanly when the user types 'quit'.

STEP 2 • TEST & DEBUG (WITHOUT AI)

Then, working without AI, test the code and find at least one bug, limitation, or improvement in the AI's solution. Fix it yourself. Common issues to look for: chunks that split mid-sentence producing broken context; no handling when FAISS returns fewer results than k (e.g. index has only 1 chunk); a prompt that does not explicitly instruct the LLM to answer only from the provided context; or the loop not stripping whitespace before comparing the input to 'quit'.

SUBMIT

All 3 items required

- 1 The exact prompt(s) you gave the AI tool.
- 2 The AI's original code and your corrected version.
- 3 A 3–4 line note explaining what you changed and why the AI's version needed it.